

SCIENCE OF SCIENCE ・ 科学計量学

OpenAlex で手を動かして学ぶ 科学計量学 入門

6つの Jupyter ノートブックで、API 取得・基礎統計・研究者予測・分野ネットワーク・ジェンダー分析・
Disruption Index を実データで体験する

題材: バクテリオファージ研究 (T11048) / 切替: 量子情報 (T10020)

このチュートリアル全体像

OpenAlex の無料 API から取得した 1 つの研究テーマを、6 つのノートブックで多角的に分析する

題材はバクテリオファージ研究というひとつのトピックです。各ノート冒頭の NAME を 'qc' に変えれば、量子情報のデータセットへ丸ごと切り替わります。各ノートは前のノートが保存したデータを読むだけで完結します。

01 OpenAlex API アクセス | 検索・絞り込み・ページング・集計といった API の作法を一通り学び、以降すべての土台となる 2 つの表（論文単位／著者 × 論文単位）を parquet に保存します。

02 基礎統計 | 被引用の極端な偏りをジニ係数とローレンツ曲線で測り、べき乗則かどうかを検定し、国際共著と埋め込みによる意味的な近傍検索まで扱います。

03 研究者の将来予測 | H-index など研究者を測る指標を比較し、最大ヒットが出る時期を検証、初期キャリアから将来の H-index を Random Forest で予測します。

04 分野ネットワーク俯瞰 | 引用グラフを Leiden 法でサブ分野に分割し、UMAP で分野の俯瞰地図を描いて、構造（引用のつながり）と内容（テキスト）という 2 つの見方を比べます。

05 ジェンダー分析 | 平均に頼らない頑健な指標と、参入年をそろえたコホート比較によって、見かけの男女差から交絡（参入時期の偏り）を切り分けます。

06 Disruption Index | 論文の革新性を「それ以前の文脈と断絶する度合い」として引用構造から測り、チームサイズが小さいほど破壊的、という有名な結果を再現します。

上から順に実行すれば、**科学計量学の代表的な分析を一通り体験できる構成です。**

Science of Science（科学計量学）とは

科学そのものを、エンティティのネットワークとして定量的に分析する学問

- 科学を「論文・著者・機関・助成金・雑誌」の巨大ネットワークとして捉え、その構造とダイナミクスを測る分野
 - 呼称 : science of science / scientometrics（科学計量学） / bibliometrics
- 代表的な問い
 - 引用はなぜ極端に偏るのか（少数の論文が引用を独占）
 - 研究者の「最高傑作」はキャリアのいつ出るのか
 - チームの大きさは研究の革新性にどう効くのか
 - 分野はどんなサブ領域に分かれ、どう発展するのか
- 本チュートリアルは、これらの定番分析を実データで一通り再現する

データはどこから来るか

商用データベース

Web of Science (Clarivate)、Scopus (Elsevier)。高品質だが有償

オープンアクセス DB

OpenAlex、Crossref、Semantic Scholar、PubMed。無償
・ API 公開

本チュートリアルの選択 : OpenAlex

完全無償・認証キー不要・API が扱いやすい

OpenAlex — 学術世界のオープンな地図

旧 Microsoft Academic Graph の後継として 2022 年に公開された無償の学術 DB

約 2.6 億

収録論文数 (Works)

6

主要エンティティ

4 階層

トピック分類

¥0

利用料・キーは不要

主なエンティティとエンドポイント

エンティティ	内容	エンドポイント
Works	論文・書籍・データセット等の成果物	/works
Authors	著者	/authors
Sources	雑誌・リポジトリ等の発行元	/sources
Institutions	研究機関（ROR 紐付き）	/institutions
Topics	研究テーマ（domain>field>subfield>topic）	/topics
Funders	助成機関	/funders

① Google Colab でワンクリック実行

各ノート最初のセルが、環境差を自動で吸収する

- バッジから開いて上から実行するだけ。インストール不要
 - 'google.colab' in sys.modules で実行環境を自動判定
 - Colab のときだけ pip install と git clone を実行
- 既定は Google Drive を使わない（権限プロンプトなし）
 - 必要データは GitHub Release から自動取得 → 完全ワンクリック
 - セッションをまたいで保存したい人だけ USE_DRIVE=True
- ensure_data(): 無ければ落として展開する冪等な取得関数
 - if os.path.exists(path): continue で二重 DL を防ぐ

```
setup cell (抜粋)

# 実行環境を自動判定して吸収する
import os, sys
USE_DRIVE = False    # 既定 : Drive を使わない

if 'google.colab' in sys.modules:
    !pip -q install requests pandas pyarrow \
        numpy matplotlib scikit-learn ...
    BASE = '/content/sciscitutorial'
    if not os.path.exists(f'{BASE}/.git'):
        !git clone -q https://github.com/\
            asatani/sciscitutorial.git {BASE}
    os.chdir(BASE)

# code/ から起動したらプロジェクト直下へ
if os.path.basename(os.getcwd()) == 'code':
    os.chdir('..')
```

② ローカル環境で動かす

Jupyter / VS Code。依存は requirements.txt にまとめて管理

ターミナル

```
git clone https://github.com/asatani/\
  sciscitutorial.git
cd sciscitutorial
python -m pip install -r requirements.txt
jupyter lab # または jupyter notebook
```

- 先頭のセットアップセルが data/ を自動で用意（無ければ Release から取得）
- code/ 以下のノートを開いて上から実行するだけ

主要な依存ライブラリ

- pandas / numpy / pyarrow — 表と数値、parquet 入出力
- scikit-learn / scipy / powerlaw — 統計・機械学習
- networkx / igraph / leidenalg — ネットワーク解析
- umap-learn — 次元圧縮 / gender-guesser — 性別推定
- openai — 埋め込み・GPT 要約（任意）

OpenAI を使うセルは任意

02 の埋め込み KNN ・ 03/04 の GPT 要約のみ。使う場合だけ環境変数 OPENAI_API_KEY を設定。未設定ならそのセルは自動でスキップされ、チュートリアルは止まらない

③ データ配布と分野の切替

重い事前計算データは git に含めず GitHub Release で配布する

Release アセット	展開先	使うノート
<code>works_bacteria.zip</code>	<code>data/works/bacteria/</code>	01-05 (bacteria)
<code>works_qc.zip</code>	<code>data/works/qc/</code>	01-05 (qc)
<code>career.zip</code>	<code>data/career/</code>	03 § 3-2
<code>supplementary.zip</code>	<code>data/supplementary/</code>	06 (DI)

再生成可能なデータは git 管理せず Release へ。ノートのセットアップセルが自動取得する

分野の切替は NAME 一行

```
NAME = 'bacteria'
# T11048 バクテリオファージ (既定)

# NAME = 'qc'
# T10020 量子情報
```

02 ~ 06 は `data/works/{NAME}/` を読むだけ。
NAME を変えて再実行すれば別分野の分析に丸ごと切り替わる。

全ノートを貫く 3つの約束事

一見バラバラなテーマも、この3点を押さえれば同じ流儀で読める

API の叩き方をそろえる。 対象は filter で絞り、必要な列だけを select で受け取り、1回200件までの上限は cursor でページングして全件を集めます。リクエストに連絡先メール（mailto）を付けると polite pool に入り、安定して速く取得できます。認証キーやトークンは一切不要です。

データは2つの表に集約する。 面倒な API 処理はすべて 01 に集め、そこで works.parquet（1行＝1論文）と paper_authors.parquet（1行＝1著者×1論文）を保存します。02以降のノートはこの2表を pd.read_parquet で読むだけ。著者を主役にする分析（H-index やジェンダー）は後者を、論文を主役にする分析（引用・ネットワーク）は前者を使い分けます。

再現性を担保する。 図のスタイルを全ノートで統一し、乱数は seed を固定して結果を再現可能にします。ベクトルの行ズレは OpenAlex ID をキーに整列して防ぎ、要所に assert を置いて『改造したときに静かに壊れる』事故を検知します。

01

NOTEBOOK 01

OpenAlex API アクセス入門

OpenAlex API access

科学計量学の第一歩は「論文データを手に入れること」。API の使い方そのものを学びつつ、全ノート
の基盤データを2つの表に整えて保存する。

規模とトピックの4階層を掴む

個別トピックに潜る前に、OpenAlex の分類体系を俯瞰する

- **per-page=1** で問い合わせると、本体を受け取らず **meta.count** (該当件数) だけを安価に取得できる
- **研究テーマは4階層で整理される**
 - domain (4) > field (26) > subfield (~250) > topic (~4,500)
 - 題材 T11048 は最下層の topic
 - domain=Physical Sciences / field=Environmental Science / subfield=Ecology
- **topic 単位で絞ると、分野横断のノイズを抑えて「ひとつの研究テーマ」を深く分析できる**

階層の確認

```
# 総数だけを安価に取得 (本体は返さない)
def oa_count(entity, filter=None):
    params = {'per_page': 1}
    if filter: params['filter'] = filter
    return oa_get(entity, **params)['meta']['count']

# 対象トピックの階層を確認
topic = oa_get(f'topics/{TOPIC}')
print(topic['domain']['display_name'])
print(topic['field']['display_name'])
print(topic['subfield']['display_name'])
print(topic['works_count']) # topic の論文数
```

全エンドポイント共通の入口 oa_get

依存を増やさず requests で直接 API を叩く。mailto を自動付与

共通ヘルパー

```
BASE = 'https://api.openalex.org'
```

```
def oa_get(entity, **params):  
    # 'works', 'authors' や 'works/W123' を叩いて JSON  
    # per_page= を API の per-page= に読み替える  
    if 'per_page' in params:  
        params['per-page'] = params.pop('per_page')  
    params.setdefault('mailto', MAILTO) # polite pool  
    r = requests.get(f'{BASE}/{entity}',  
                    params=params, timeout=60)  
    r.raise_for_status()  
    return r.json()
```

- 1つの関数で works / authors / institutions ... すべてを叩ける
- mailto を付けると polite pool に入り、安定して速く取得できる
- Python のキーワード引数に '-' が使えないため per_page を per-page に自動変換
- raise_for_status() で HTTP エラーを早期に検知
- 認証キーは一切不要

まず 1 件だけ取得して論文の構造を見る

一括取得の前に、Work 1 件の JSON でフィールドを掴む

- 各エンティティは URL 末尾の ID で直接引ける（例 /works/W2741809807）
- 返り値は深くネストした辞書。title・year・cited_by_count・authorships・referenced_works など
- 抄録は著作権上の理由で **inverted index** で返る
 - inverted index = 語 → 出現位置のマップ
 - 位置順に語を並べ直せば通常の文章に復元できる

抄録の復元

```
# 抄録 (inverted index) を文章に戻す
def abstract_text(inverted_index):
    if not inverted_index:
        return ''
    # 語 -> 位置リスト を 位置 -> 語 に反転
    word_at = {pos: word
                for word, positions in inverted_index.items()
                for pos in positions}
    # 位置順に語を並べる
    return ' '.join(word_at[i]
                     for i in sorted(word_at))
```

cursor ページングで全件を集める

API は 1 回で最大 200 件。next_cursor を辿って全ページを回収

cursor ページング

```
def fetch_works(topic, to_date='2024-12-31',
                max_records=200000):
    params = dict(
        filter=f'primary_topic.id:{topic}',
        f'to_publication_date:{to_date}',
        sort='cited_by_count:desc',
        per_page=200, cursor='*',
        select='id,doi,cited_by_count,fwci,...')
    works = []
    while len(works) < max_records:
        r = oa_get('works', **params)
        works += r['results']
        cursor = r['meta'].get('next_cursor')
        if not cursor or not r['results']:
            break
        params['cursor'] = cursor
    return works[:max_records]
```

- filter — トピックで限定し、to_publication_date で直近年を除外
 - 直近年は引用が未成熟。含めると被引用比較が不公平に
- sort=cited_by_count:desc — 影響の大きい論文から代表的に拾える
- select — 必要な列だけ指定し転送量を削減
- cursor — page= 方式は 1 万件まで。大量取得は cursor 一択

チュートリアルの中台となる 2 つの表

面倒な API 処理は 01 に集約。02 以降は `pd.read_parquet` で読むだけ

works.parquet (1 行 = 1 論文)

- cited_by_count — 被引用数 (インパクト)
- fwci — 分野・年で正規化した被引用 (1.0= 世界平均)
- n_authors / countries / institution_ids — チームと所属
- referenced_works — 参考文献 ID (引用ネットワークの素)
- abstract — inverted index から復元した抄録

paper_authors.parquet (1 行 = 1 著者 × 1 論文)

- author_id / author_name — 著者 (H-index・リーダーボード)
- author_position — first / middle / last (貢献の目安)
- institutions / countries — その論文時点の所属

⚠ 著者の重複計上に注意

OpenAlex は所属機関ごとに同一著者を複数計上することがある。集計前に `drop_duplicates(['author_id','work_id'])` が必須 (放置すると H-index が論文数を超える)

ID が分からないときはテキスト検索

keyword・著者・機関・雑誌を同じ流儀で探す

- **search=...** — タイトル・抄録・全文を横断し関連度順に返す
- **filter=title.search:... / abstract.search:...** — 対象を限定し、年・被引用・OAなど他条件とANDで組める
 - filter内は, がAND、| がOR
- **著者: /authors?search= 氏名 → /works?**
filter=author.id:A...
- **機関: /institutions?search= 名 → filter=institutions.id:I...**
 - 同姓同名は works_count・所属で本人を見極める

検索の組み合わせ

```
# タイトル限定 + 追加フィルタを AND (,) で連結
res = oa_get('works',
             filter='title.search:CRISPR, '
                  'from_publication_date:2015-01-01, '
                  'cited_by_count:>100',
             sort='cited_by_count:desc', per_page=5,
             select='id,display_name, '
                  'publication_year,cited_by_count')

# 著者 ID でその人の全業績を絞る
oa_get('works', filter=f'author.id:{aid}',
       sort='cited_by_count:desc')
```

サーバ側集計と、賢い一括取得

全件をダウンロードせずに分析を進めるための機能を押さえる

group_by で分布を安価に取る。 指定したカテゴリ（出版年・国など）ごとの件数をサーバ側で集計して返します。年次論文数や国別内訳のような分布を、本体をダウンロードすることなく 1 リクエストで得られます。02 の年次推移や 04 の集計もこの group_by を使っています。

sample と autocomplete。 sample=N&seed=... は、条件を満たす母集団から偏りのないランダム抽出を行い、seed を固定すれば同じ標本を再現できます（03 §3-2 のランダム著者はこの方法で選びます）。autocomplete は入力補完用の軽量検索で、機関や ID を手早く探すときに便利です。

OR フィルタでまとめ取り。 著者 ID を author.id:A|B|... のようにパイプで区切ると、1 リクエストで最大 50 人分の業績をまとめて取得できます。付録のランダム著者取得では、これによって数千回のリクエストを数十バッチにまで削減しています。

02

NOTEBOOK 02

基礎統計を見る

Basic statistics of the corpus

01 が保存したコーパスを読み、規模・被引用の偏り・裾分布の検定・国際共著・テキスト分析まで、科学計量学の定番の記述統計を一通り確認する。

平均だけでは見えないこと

describe と年次集計から、4つの経験則を読み取る

● 被引用は 平均 ≫ 中央値

分布が強く右に歪む。少数の超高被引用論文が平均を押し上げる

● 近年ほど平均被引用が小さい

引用が貯まる時間が短いだけ。質が低いわけではない（時間効果）

● 著者数は経年で増加

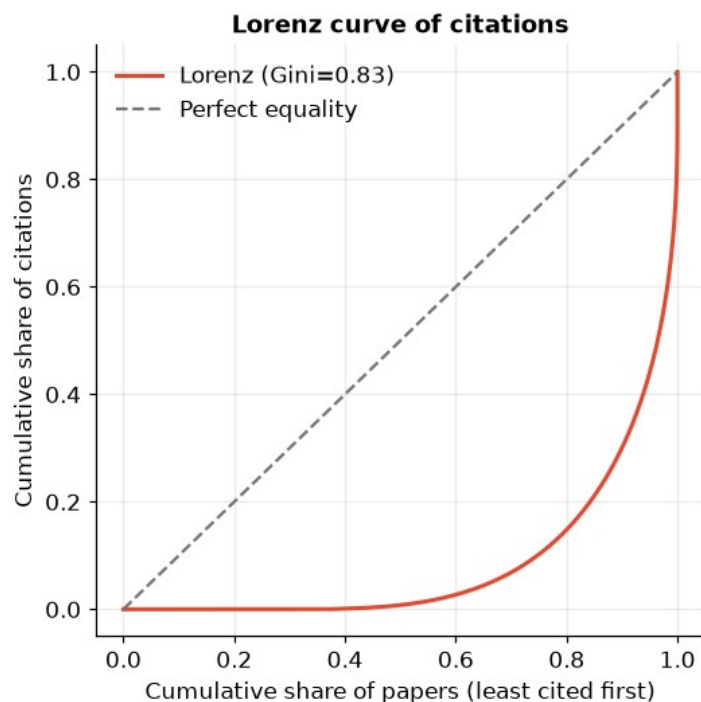
研究のチーム化・大型化を反映する

● OA 率が上昇

オープンアクセス出版の普及を示す

ジニ係数とローレンツ曲線

「ごく一部の論文が引用の大半を集める」を定量化する



ノートブック出力: ローレンツ曲線 (Gini=0.83)

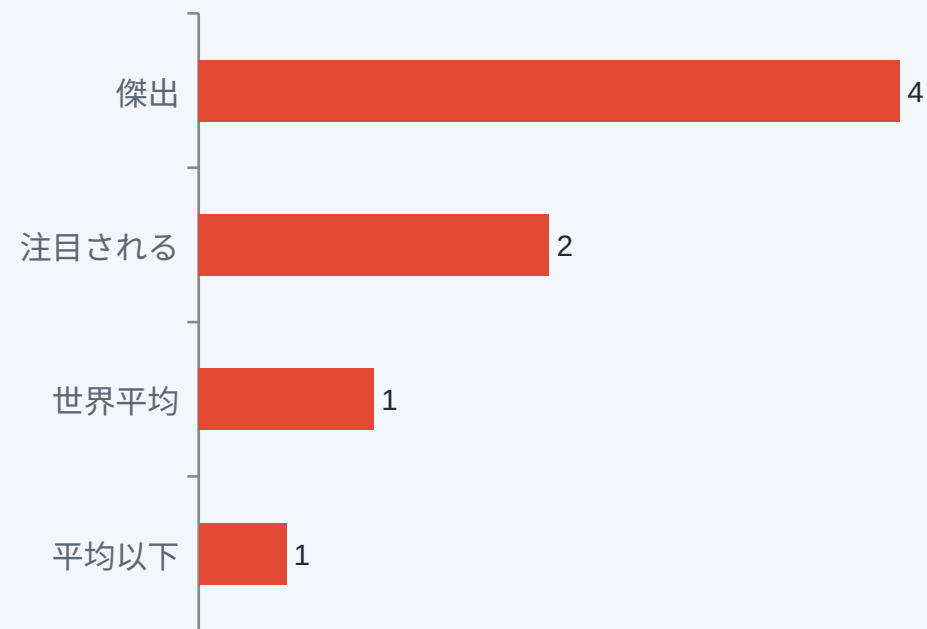
- ジニ係数 = 0 (全論文が均等) ~ 1 (1本が独占)。所得格差の指標としても使われる
- **本データのジニは 0.8 超 — 多くの国の所得格差よりはるかに偏る**
- 上位 1% の論文だけで全引用のかなりの割合を占め、多くの論文はほとんど引用されない
- **含意: 被引用をそのまま平均する分析は危うい**
 - FWCI・中央値・対数変換・上位 % の二値化などで偏りを扱う

FWCI — 公平にインパクトを比べる

Field-Weighted Citation Impact：分野と出版年で正規化した被引用

- 生の被引用は「分野」と「年数」で桁が変わる
 - 生物学と数学、2005年と2023年の論文をそのまま比べられない
- FWCI は同じ分野・同じ年の平均で割って正規化
 - 1.0 = 世界平均、2.0 = 平均の2倍の引用
- **本チュートリアル多くの比較で中心的に使用**
 - 国際共著 (02) ・ 研究者予測 (03) ・ ジェンダー (05) で活躍

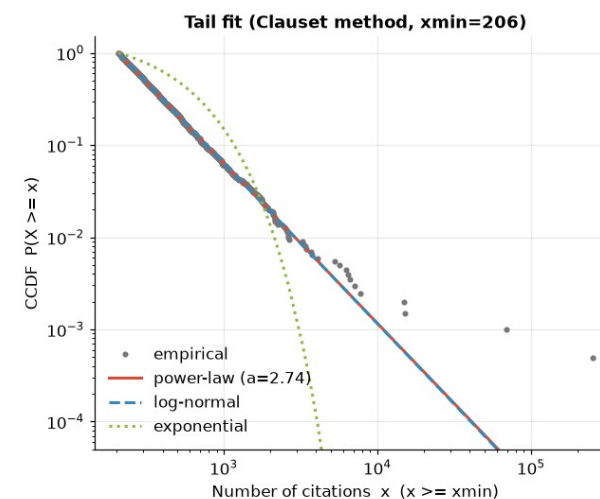
FWCI の読み方



べき乗則か? — Clauset 法

両対数で直線に見えても、それだけでは power-law の証拠にならない

- log-normal など他の裾の重い分布もほぼ直線に見える
- Clauset, Shalizi & Newman (2009) の 3 手順
 - ① 下限 x_{min} を推定 (KS 距離が最小になる点)
 - ② 指数 α を最尤推定 (x_{min} 以上のデータで)
 - ③ 対立分布と尤度比検定 (Vuong) で比較
- 実装は powerlaw パッケージ (Alstott et al. 2014)



ノートブック出力: 裾の CCDF 当てはめ ($x_{min}=206, \alpha=2.74$)

R の符号で優劣、p で有意性を判断する

「べき乗則」と決めつけない

このサンプルでの尤度比検定の結論

比較	結果	解釈
power-law vs exponential	$R \gg 0, p < 0.001$	指数分布は明確に棄却
lognormal vs exponential	$R \gg 0, p < 0.001$	裾は重い (heavy-tailed)
power-law vs log-normal	$R \approx -1.6, p \approx 0.12$	統計的に区別できない
power-law vs 切断べき乗則	$R \approx -2.1, p \approx 0.004$	切断べき乗則が優位

結論 「指数分布ではない重い裾」は確実。だがべき乗則か log-normal かは断定できず、切断べき乗則が最も無難。被引用を語るときは検定で裏取りするのが安全。

国際共著を 3 つの視点で見る

関与国が 2 か国以上の論文を「国際共著」と定義して分析する

どれくらい広がっているか。 年ごとに国際共著率（intl を 0/1 とみなした平均）を取ると右肩上がりで、共同研究の国際化が進んでいることが分かります。ただし 1 論文あたりの関与国数は大半が 1 ～ 2 か国に集中し、多国籍の大型共著はまれです。

インパクトはどう違うか。 国内群と国際群でインパクトの分布を比べます。被引用は極端に歪むため平均ではなく中央値・箱ひげで比較し、分野と年で正規化済みの FWCI も併用します。いずれの指標でも国際共著群のほうが高い傾向が見えます。

何が国際共著率を左右するか。 1 論文を explode で「論文 × 国」に展開して国ごとの率を出すと、研究規模の小さい国ほど高く、自国内で完結しやすい大国ほど低く出ます。またチーム（著者数）が大きいほど国際化が進みます。

ただし国際共著の高インパクトには、テーマの規模や資金といった交絡も含まれる。相関であって因果ではない点に注意する。

埋め込みで「意味的に近い論文」を探す

OpenAI text-embedding-3-small (256 次元) + cosine 類似度

- **タイトル+抄録を埋め込み、cosine 類似度で近傍を検索**
 - この埋め込みは 04 の俯瞰地図でも再利用する
- **OpenAlex ID 付きで保存して行ズレを防ぐ**
 - embeddings.npz に ids と vecs を一緒に保存
 - 読込時は short_id をキーに works_df の行順へ整列
 - 並べ替え・再取得しても対応がずれない
- **OPENAI_API_KEY が必要（未設定ならスキップ）**

ID 整列した埋め込み

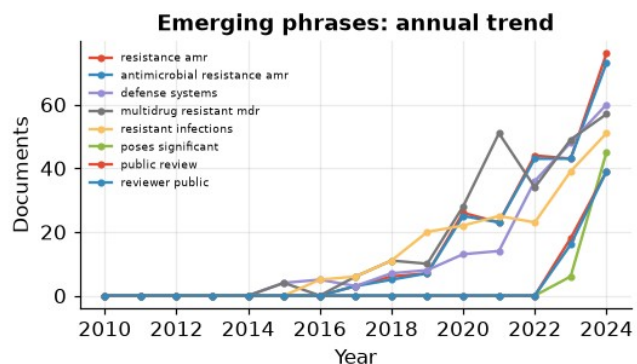
```
# ID 付きで保存 → 行がずれても再整列できる
np.savez(EMB_PATH,
         ids=sub['short_id'].to_numpy(),
         vecs=vecs)

# cosine 類似度で近傍検索
M = emb[rows] / np.linalg.norm(
    emb[rows], axis=1, keepdims=True)
q = emb[query] / np.linalg.norm(emb[query])
sims = M @ q
order = np.argsort(sims)[::-1][1:7] # 上位6
```


N-gram で「今きている言葉」を見つける

年ごとにタイトル+抄録の bi/tri-gram を数え、近年に増えた語を抽出

- `CountVectorizer(ngram_range=(2,3))` で 2 ～ 3 語の連語を集計
- 出版社ボイラープレート や MathML タグを除外
- 初出 $\geq$ 2015 かつ直近年の頻度が閾値以上を『新興』とする



ノートブック出力: 新興フレーズの年次トレンド

02 のまとめ

- 被引用は右に強く歪み、近年ほど貯まりにくい（時間効果）
- ジニ 0.8 超 — 平均する分析の危うさ。裾は Clauset 法で検定
- 国際共著は増加傾向で高インパクト（相関 $\neq$ 因果）
- 埋め込み KNN と N-gram でテキストからも分野を掴む

03

NOTEBOOK 03

研究者を数値化して将来を予測する

Quantifying researchers & prediction

1 人の研究者の業績をどう数値で測り、初期のキャリアから将来をどこまで予測できるか。指標の比較・ランダムインパクトルール・Random Forest 予測を扱う。

研究者をどう測るか — 4つの指標

1つの数字で測ろうとすると、指標ごとに見えるものと弱点が変わる

指標	測れるもの	弱点
発表論文数	生産力（どれだけ書いたか）	影響力を無視（多産でも引用0かも）
総被引用数	影響力の総量	1本の大ヒットに引きずられる
平均被引用数	1本あたりの質	多産な人が不利／少数だと不安定
h-index	量と質のバランス・持続的な影響	分野差・キャリア長依存、上限は論文数

h-index の定義：被引用が h 以上の論文が h 本ある、を満たす最大の h 。量と質を1つにまとめた定番指標

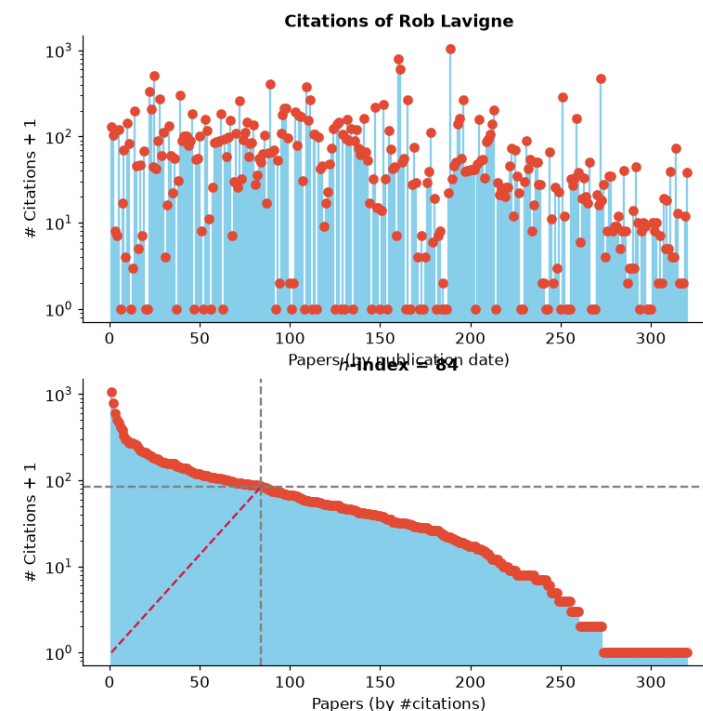
H-index を stem plot で図解する

被引用降順に並べ、45°線（被引用＝順位）を上回る最大順位が h

h-index の計算

```
def h_index(citations):
    # 被引用の多い順に並べる
    c = np.sort(np.asarray(citations))[:, -1]
    # 被引用 >= 順位 を満たす本数
    return int((c >= np.arange(
        1, len(c)+1)).sum())
```

- リーダーボード上位の研究者で、論文を被引用降順に並べて可視化
- 45°線とhのガイド線を重ねると、hの意味が一目で分かる



ノートブック出力：被引用降順の stem plot と 45°線（h-index）

リーダーボードと活動要約

H-index 分布・日本 / 国別比較・研究テーマの自動要約

リーダーボード

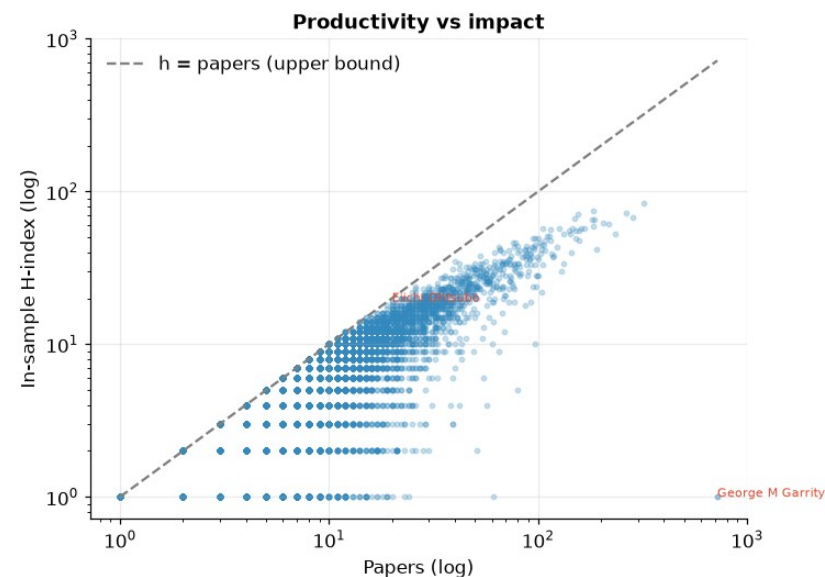
著者ごとに論文数・総被引用・H-indexを集計。分布はロング裾。日本 (JP) 主所属や国別（中央値 / トップ H-index）でも比較

生産性 vs インパクト

論文数（生産性）と H-index（持続的インパクト）を両対数で散布。右下 = 多作だが低インパクト、左上 = 少数精鋭が読める

活動要約 (TF-IDF/GPT)

各研究者の抄録に TF-IDF で特徴語を抽出。過去 / 最近のトップ論文と併せ「何を研究してきた人か」を俯瞰。任意で GPT 要約



ノートブック出力: 生産性 vs インパクト

トップ研究者の活動を GPT で解説する

代表論文（最近 5 件／昔 5 件）を渡し、研究テーマの中心と変遷を日本語で要約させる

手順。 リーダーボード 1 位の著者を選び、被引用トップ 5 を「最近（ ≥ 2020 ）」と「昔（ < 2020 ）」に分けてタイトルを列挙。プロンプトに埋めて gpt-5.4-mini に渡す（キーが無ければプロンプトだけ表示）。

GPT で著者を要約

```
recent5 = top5_titles(papers[year >= 2020])
old5     = top5_titles(papers[year < 2020])
prompt = (f" 研究者「{name}」の代表論文です。 \n"
          f" 【最近のトップ 5】 \n{recent5}\n"
          f" 【昔のトップ 5】 \n{old5}\n"
          " 研究テーマの中心と、昔から最近への "
          " 変遷を日本語 200 字程度で解説して。 ")
resp = OpenAI().chat.completions.create(
    model=GPT_MODEL,
    messages=[{"role": "user", "content": prompt}])
print(resp.choices[0].message.content)
```

GPT の出力（研究者：Rob Lavigne）

研究の中心は、**バクテリオファージの分類・ゲノム情報学・応用（治療／農業）**。

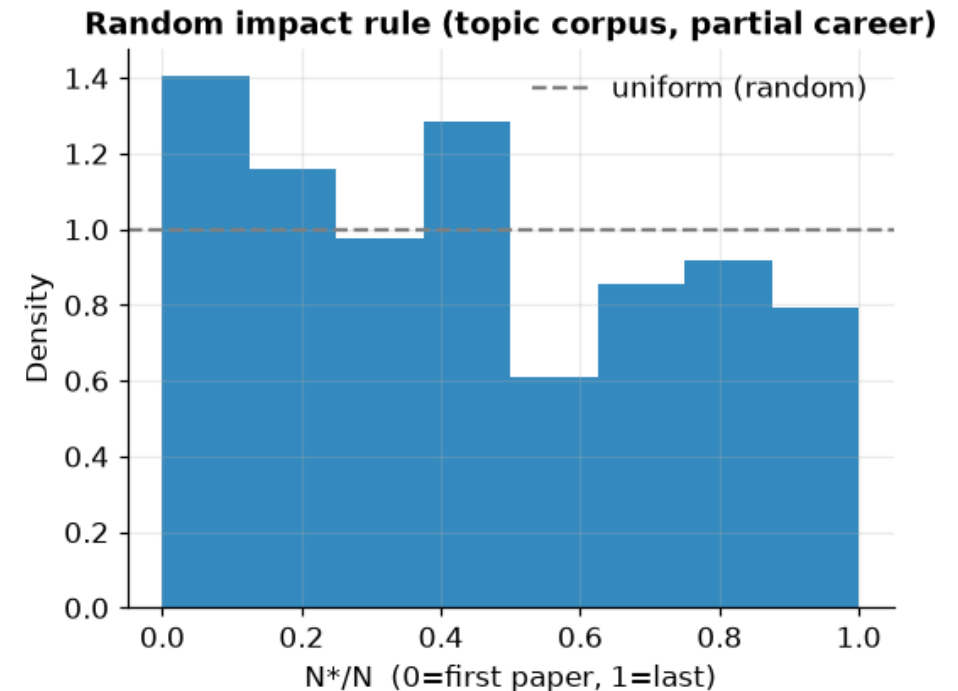
昔は、未培養ウイルスのゲノム分類法や MIUViG、宿主認識、精製カクテルやエンドリシンなど、基盤技術とファージ療法の土台づくりが主軸。

最近は、ICTV の分類更新、耐性菌に対するファージ＋抗菌薬の実治療、植物保護へのバイオコントロールへと比重が移り、基礎研究から実装・標準化へ発展。

ランダムインパクトルール

Sinatra et al. (2016): 最大ヒットが出るタイミングはキャリア中でランダム

- 研究者の「最高傑作」が出る位置は、論文を出版順に並べた系列の中でほぼ一様分布
- 「大器晩成型」も「若くしてピーク型」も特別ではない — 最高傑作はキャリアのどこにでも等確率で現れる
- 検証：各著者の最大ヒットが何本目か（相対順位 N^*/N ）
 - 0 = 処女作、1 = 最終作
 - その分布が一様かを KS 検定で確かめる

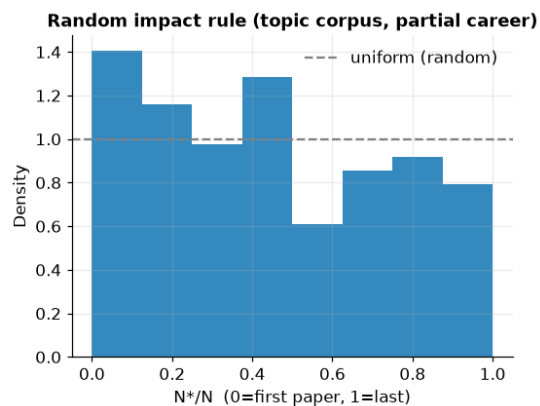


ノートブック出力: N^*/N の分布 (§3-1 トピック内、一様に近い)

データセット内は初期に偏り、全業績ではほぼ均等

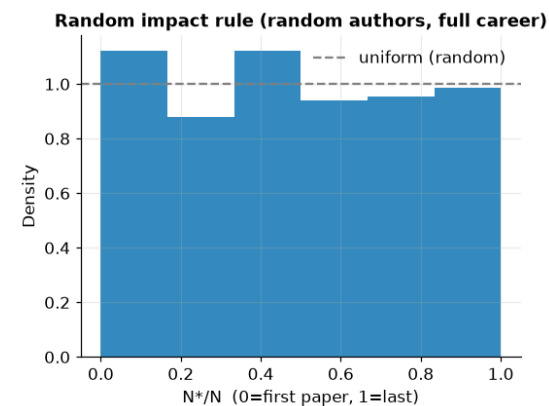
同じ経験則を「トピック内」と「全業績」の2通りで確かめる

§3-1 トピック内（部分キャリア）



1 トピックに限ると各著者は部分キャリア。最大ヒットは前半に偏る (mean < 0.5)

§3-2 ランダム著者の全業績（完全版）



全業績で見るとほぼ一様。KS 検定でも一様性は棄却されない＝元論文どおり

初期キャリアから特徴量を作る

最初の出版から EARLY_YEARS 年以内を early window とする

- 貢献の特徴量

- early_papers / early_citations / early_h
- early_first_share ・ early_last_share （著者順位）

- 研究環境の特徴量

- early_mean_coauthors （共著者数）
- early_intl （国際共著率） ・ early_inst_div （機関多様性）

- 将来アウトカム = early 以降の論文数・引用・H-index

early / future の分割

```
# キャリア年齢で初期 / 将来 に分ける
pa['first_year'] = pa.groupby(
    'author_id')['publication_year'].transform('min')
pa['career_age'] = (pa['publication_year']
                    - pa['first_year'])

early = pa[pa['career_age'] < EARLY_YEARS]
future = pa[pa['career_age'] >= EARLY_YEARS]

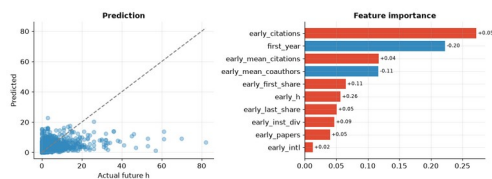
# early から特徴量、future からアウトカムを集計
features = early.groupby('author_id').agg(...)
```

将来 H-index を Random Forest で予測

初期数年の特徴量から将来を予測し、どの初期指標が効くかを見る

学習と予測

```
model = make_pipeline(
    SimpleImputer(strategy='median'),
    RandomForestRegressor(n_estimators=300,
                          min_samples_leaf=3, random_state=42))
model.fit(train[features], train['future_h'])
pred = model.predict(test[features])
```



ノートブック出力: 予測精度と特徴量重要度

⚠ 教材としての注意

- cited_by_count は「取得時点」の累積値で、過去の各時点の引用ではない
- 厳密な将来予測には各時点のスナップショットが必要
- ここでは相関構造を見る教材として扱う（因果ではない）

早期業績と将来アウトカムの相関

散布図に Pearson r ・ p 値を注記（因果ではなく相関構造の確認）

- **early_citations → future_citations**
- **early_h → future_h**、**early_papers → future_papers**
 - 歪み対策に平方根変換、対角線は $y=x$
- **初期のインパクト・H-index は将来と正の相関**
 - 早く良い成果を出す人ほど後も伸びやすい傾向
 - Matthew 効果（富める者がさらに富む）と整合的

03 のまとめ

- 4 指標の長所と弱点を理解し、h-index を図解
- 最大ヒットの出現はキャリア中でランダム（Sinatra 2016）
- 初期キャリアから将来 H-index を Random Forest で予測
- 早期の成果は将来と相関（相関≠因果に注意）

04

NOTEBOOK 04

研究分野をネットワークで俯瞰する

A bird's-eye map of a field

論文どうしの引用関係（ネットワーク）と、抄録の内容（テキスト）という2つの見方で、分野の内部構造を「俯瞰地図」として可視化する。

分野を俯瞰する 2 つの見方

「引用のつながり」と「内容の近さ」を同じ地図の上で比べる

分野の内部構造を俯瞰するには、大きく 2 つの見方があります。この章では両方を同じ 2 次元地図の上に描き、どれだけ一致し、どこでズレるかを見比べます。

引用のつながり（構造）。 「誰が誰を踏まえているか」という関係です。論文どうしの直接引用からグラフを作り、Leiden 法で密に結ばれたグループ（サブ分野）に分割します。これは、互いに引用し合う閉じた研究コミュニティのまとまりを表します。

内容の近さ（テキスト）。 「何について書いているか」という意味です。タイトルと抄録を埋め込みや TF-IDF でベクトル化し、UMAP で 2 次元に圧縮すると、意味的に近い論文どうしが地図上で近くに集まります。

両者を重ねて読む。 同じ地図を 2 つの色分けで並べると、よく一致する領域は「引用も内容も閉じたコミュニティ」、ズレる領域は「手法は共有しつつ別々に引用される領域」と解釈できます。

直接引用ネットワークを作る

被引用上位コア (NET_WORKS=30000) に絞って可読性を確保

引用グラフの構築

```
ids = set(works_df['id'])
G = nx.Graph()
for wid, refs in zip(works_df['id'],
                    works_df['referenced_works']):
    for r in set(refs):
        # 相手もコア集合内 かつ 自己引用でない
        if r in ids and r != wid:
            if G.has_edge(wid, r):
                G[wid][r]['weight'] += 1 # 相互引用
            else:
                G.add_edge(wid, r, weight=1)

# 最大連結成分だけ残す
main = G.subgraph(max(
    nx.connected_components(G), key=len)).copy()
```

- 参考文献に相手 ID が含まれればエッジを張る
- 全件 (約 10 万) だと密なハブになり地図が読めない → 上位コアに絞る
- works.parquet は被引用降順なので .head() を取るだけ
- 相互引用は weight=2 として結びつきの強さを表現
- 孤立した断片を落とし、主要な塊に解析を集中

Leiden 法でサブ分野に分割

辺で強く結ばれた論文グループ（＝サブ分野）に自動分割する

- **igraph + leidenalg で高速にコミュニティ検出**
 - Leiden 法 = Louvain を改良し、途切れないコミュニティを保証（Traag et al. 2019）
- **resolution_parameter で粒度を調整**
 - 大きいほど細かく、小さいほど粗い塊に
- **seed=42 と ノード順の固定 で再現性を確保**
 - Leiden は内部で乱択を使うため
- **本データでは 約 29 個のコミュニティを検出**

Leiden コミュニティ検出

```
nodes = sorted(main.nodes()) # 順序固定
g = ig.Graph(n=len(nodes), edges=[...])
g.es['weight'] = [...]

partition = leidenalg.find_partition(
    g, leidenalg.RBConfigurationVertexPartition,
    weights='weight',
    resolution_parameter=1.6, seed=42)
```

サブ分野に名前を付ける（識別語 → GPT）

まず contrast 方式で識別語を取り出し、それを入力に GPT が短い英語の分野名を返す

入力。 各サブ分野の識別語（クラスタ平均－全体平均が大きい語。汎用語や phage 等の頻出語は除外）。

処理と結果。 識別語を GPT に渡し、5 語以内の英語の分野名を JSON で受け取る。API キーが無ければ上位 2 語で代用する。

GPT による命名

```
prompt = ('Return a JSON object mapping cluster '
          'IDs to short English field names '
          '(max 5 words).\n' + keywords_per_cluster)
resp = OpenAI().chat.completions.create(
    model=GPT_MODEL, temperature=0,
    response_format={'type': 'json_object'},
    messages=[{'role': 'user', 'content': prompt}])
names.update(json.loads(
    resp.choices[0].message.content))
```

クラスタの識別語（入力）	GPT の分野名（結果）
t4, t2, t5, synthesis, typing	Phage typing
therapy, infections, resistant	Phage therapy
lambda, mu, p1, p22, repressor	Lambda gene regulation
virome, gut, diversity, metagenomic	Gut virome diversity
assembly, capsid, self, mosaic	Capsid self-assembly
marine, abundance, sea, aquatic	Marine cyanophages
endolysin, lysis, endolysins, lysin	Endolysin enzymes
defense, systems, toxin, antitoxin	Bacterial defense systems

その前に — 埋め込みという特徴量

地図を描く前に、論文をどう数値ベクトルにするかを定める

- 各論文を「内容を表す数値ベクトル」にして初めて地図が描ける
- 02 で作った OpenAI 埋め込み（text-embedding-3-small, 256 次元）を再利用
 - embeddings.npz を short_id で works_df に整列（順序ズレに強い）
 - 埋め込みのある論文だけに絞ってから地図化
- この埋め込みから地図を 2 通り描く
 - 地図①: 256 次元の埋め込みをそのまま UMAP に入れる
 - 地図②: SVD で 50 次元に圧縮してから UMAP に入れる
- 類似度は cosine（角度）で測るのが定石



地図① 生の埋め込み（256 次元）

意味をそのまま捉える密ベクトル。文脈や言い換えに強いが計算はやや重い

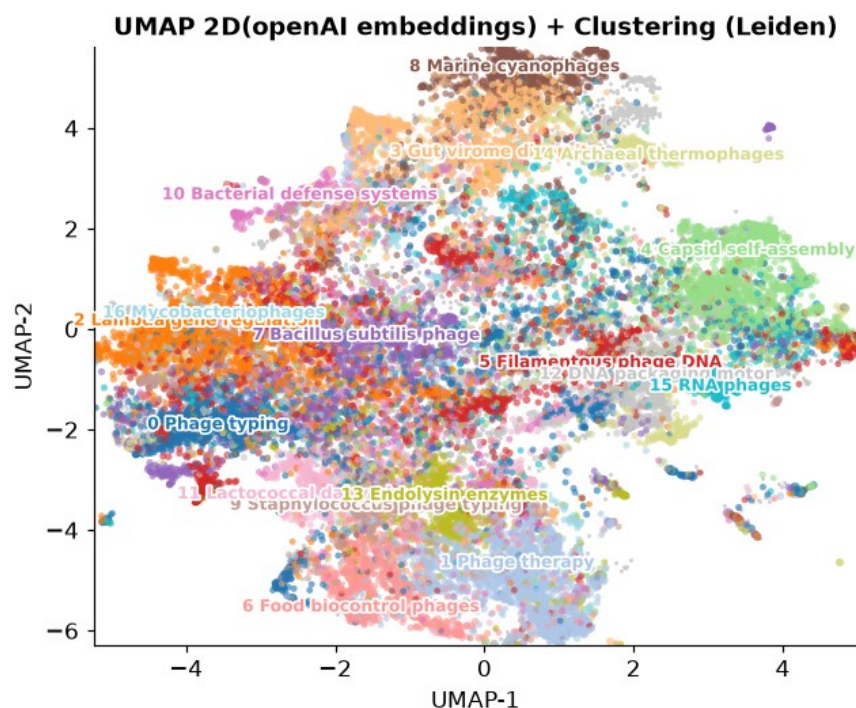


地図② 埋め込み + SVD（50 次元）

主要な軸だけに圧縮。軽くノイズも減る。大域構造が保たれるかを地図①と見比べる

地図①: OpenAI 埋め込みの俯瞰図

UMAP で2次元化し、Leiden サブ分野を色分け・§4でGPTが付けた分野名を重心に注記



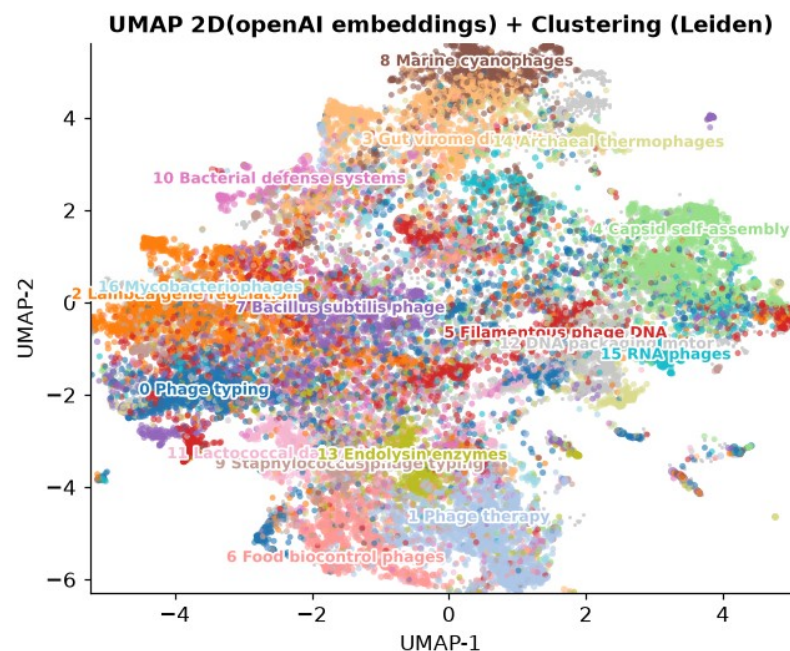
ノートブック出力: UMAP(OpenAI 埋め込み) + Leiden。各島に GPT 命名の分野名

- 似たテーマの論文が近くに集まり、色の島がサブ分野になる
- 各クラスターの重心に、GPT が付けた英語の分野名を注記
 - Phage therapy / Marine cyanophages / Capsid self-assembly / Gut virome diversity ...
- 点の大きさは被引用数 ($\sqrt{\text{スケール}}$) で、影響の大きい論文ほど大きい
- `n_neighbors`・`min_dist`・`metric='cosine'` が地図の形を決める

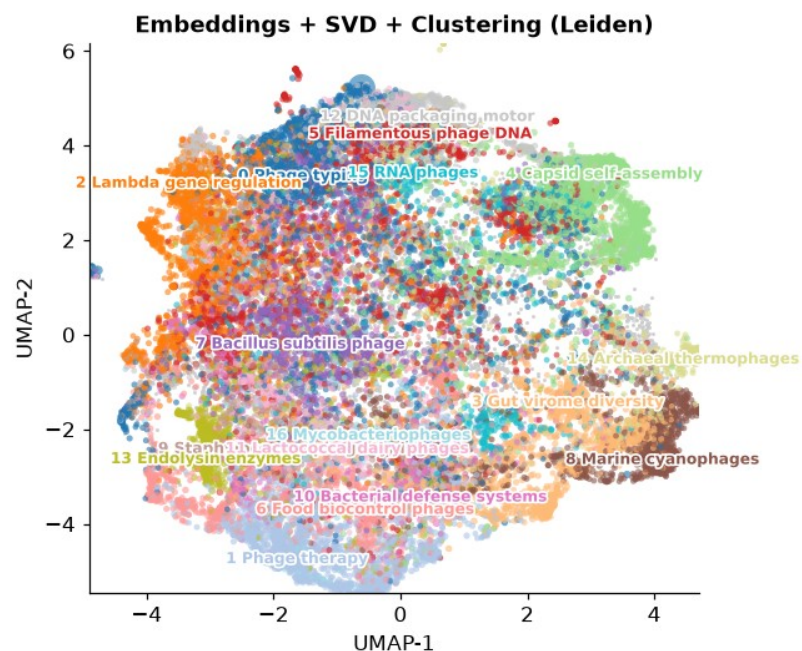
引用: McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*.

地図②：埋め込みを SVD で圧縮しても構造は保たれる

同じ OpenAI 埋め込みを 50 次元に SVD 圧縮してから UMAP。地図①とほぼ同じ島が並ぶ



地図①: OpenAI 埋め込み (256 次元)



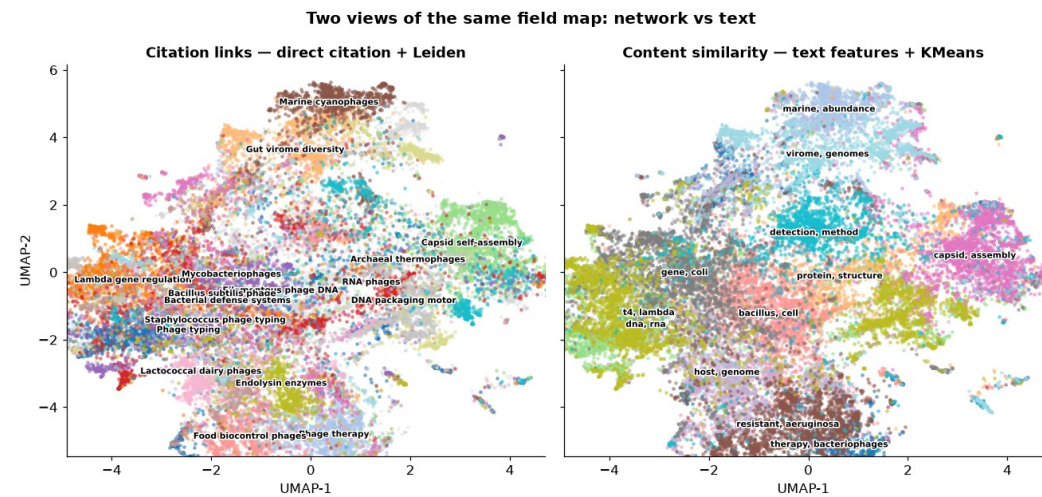
地図②: 埋め込み + SVD (50 次元)

読み方 50 次元まで圧縮しても主要なサブ分野の配置はほぼ変わらない = 埋め込みの情報は少数の軸に集約されている。軽い地図②で十分実用になる。

ネットワーク vs テキストを並べる

同じ UMAP 座標を、引用 (Leiden) 色と内容 (KMeans) 色で塗り分ける

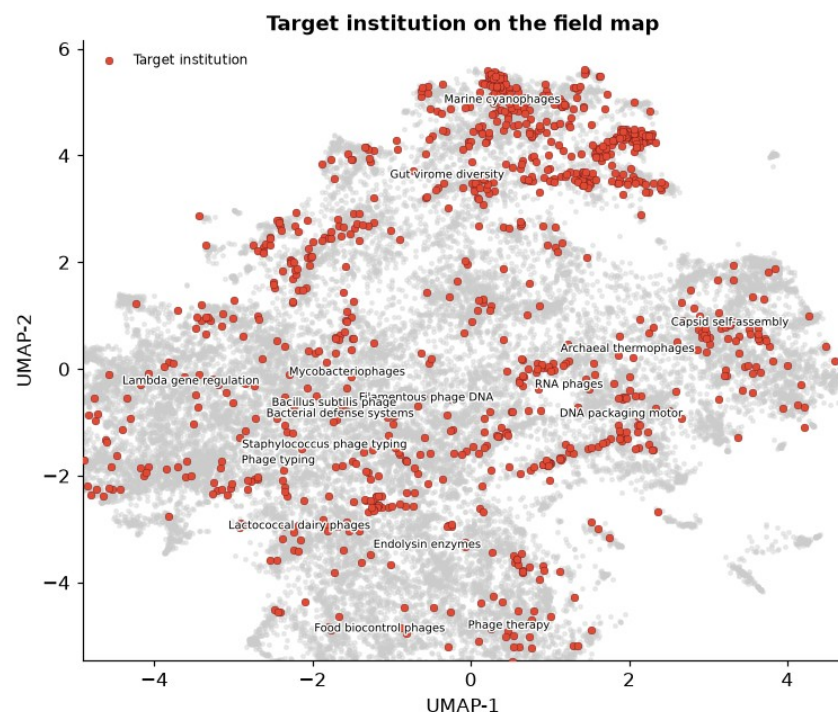
- 左：引用のつながり（直接引用 + Leiden）
- 右：内容の近さ（テキスト特徴量 + KMeans）
 - KMeans は引用関係を一切使わず、テキストだけで分割
- x/y 座標は同一 → 点の位置は固定で色だけが変わる
- 「同じ場所が両方で同じ色群になるか」を直接見比べられる



ノートブック出力: 引用 (Leiden) vs 内容 (KMeans) の二視点

機関の強みを地図で読む

institution_ids で対象機関の論文を判定し、どのサブ分野に強いかを見る



ノートブック出力: 対象機関の論文を俯瞰地図にハイライト

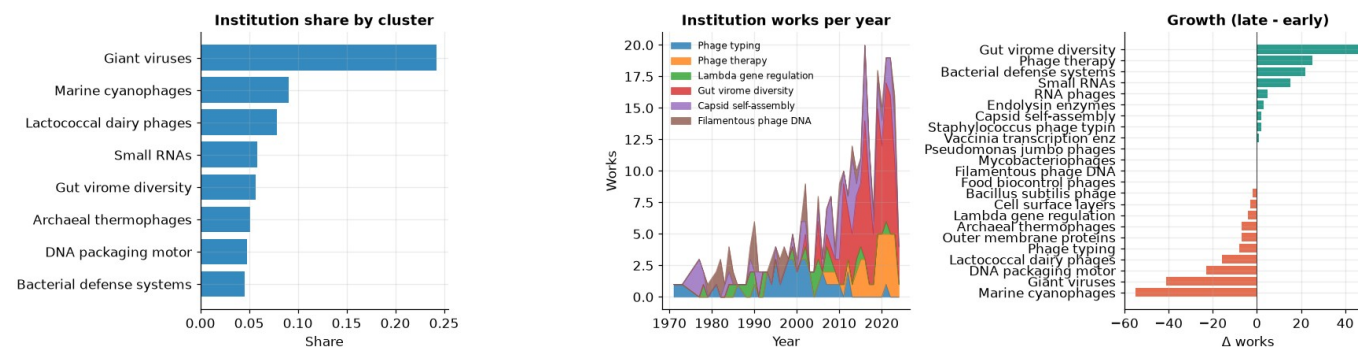
サブ分野別シェアの上位（実データ）

その分野の論文に占める、対象機関の割合

サブ分野	シェア
Giant viruses	24.2%
Marine cyanophages	9.0%
Lactococcal dairy phages	7.9%
Small RNAs	5.9%
Gut virome diversity	5.7%
Archaeal thermophages	5.1%
DNA packaging motor	4.8%

シェア・年次推移・成長率で強みを定量化

地図のハイライトを、3つの指標で数値に落とし込む



ノートブック出力: (左) サブ分野別シェア (中) 年次推移 (右) 成長率 後半-前半

04 のまとめ

- 引用グラフ → Leiden → 識別語 + GPT 命名 → UMAP で、名前付きの分野俯瞰地図を構築した
- 埋め込みは生 (256 次元) と SVD 圧縮 (50 次元) で地図を 2 枚描き、構造がほぼ保たれることを確認。構造 (引用) と内容 (テキスト) の 2 視点も並置し、機関の強みまで定量化した

05

NOTEBOOK 05

ジェンダー差を最小限の指標で見る

Gender differences, minimally

「女性はデビュー時に有力者とつながりやすい」——これは女性特有の傾向か、単に参入が最近に偏るせいかな。参入年（コホート）を揃えて交絡を切り分ける。

交絡を切り分ける

見かけの差が、本当に性別によるものかを問い直す

- 素朴な観察：女性はデビュー時に有力者となつたりやすい
- **しかし女性の参入は最近（新しいコホート）に偏っている**
 - → その「見かけの傾向」は参入時期の交絡かもしれない
- **対策：参入年（コホート）を揃えて比較する**
 - 同じ年に始めた男女だけを比べれば、時期の影響を除ける

分析の流れ（6 ステップ）

- ① 著者単位で比較
- ② 出版年ごとに比較
- ③ エントリー年（参入時期を揃える）
- ④ 共著・デビュー時の最強共著者
- ⑤ 生存率（漏れるパイプライン）
- ⑥ キャリアイヤー別 top-1%

名前ベースのジェンダー推定とその限界

gender-guesser で名（先頭トークン）から man / woman / unknown を推定

名前ベース推定

```
import gender_guesser.detector as gg
detector = gg.Detector(case_sensitive=False)
first = names.str.split().str[0]
gender = first.apply(detector.get_gender)
# male/mostly_male -> man, female... -> woman
```

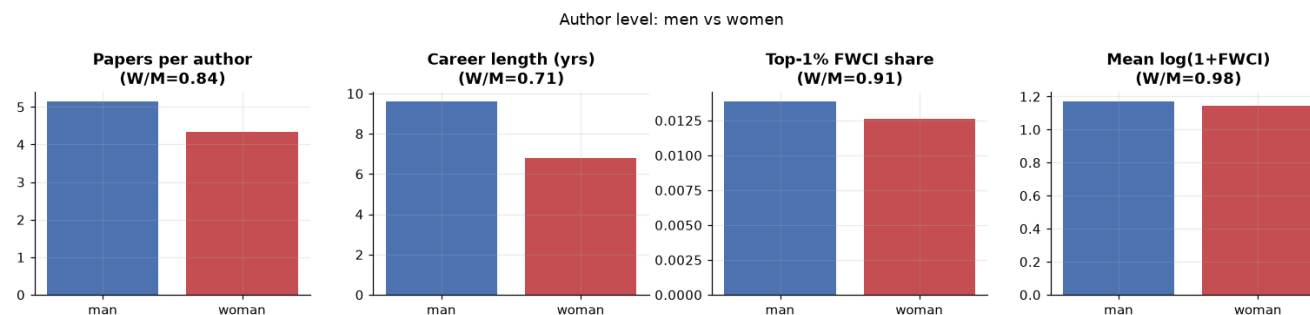
記述統計の例示であり、能力差や因果を主張するものではない

⚠ 手法の限界（必ず認識する）

- unknown が多数（名から判定できない）
- 誤分類・文化差（非英語圏の名前で顕著）
- 非二元的ジェンダーの不可視化
- 研究用途では倫理的・方法論的検証が必須

著者単位の男女差（頑健指標）

平均ではなく $\text{mean log}(1+\text{FWCI})$ と top-1% シェアで比較する



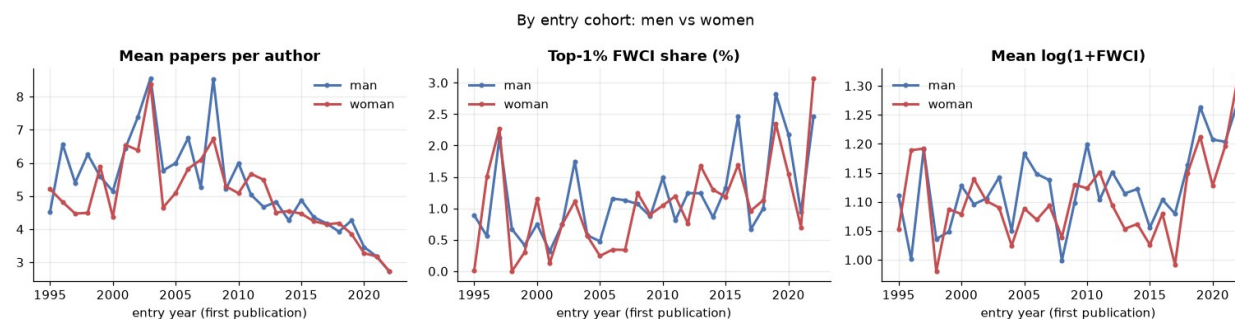
ノートブック出力：著者単位の男女比較（各図に W/M 比を併記）

読み取れること

- 論文数・キャリア長は男性が上（ $W/M < 1$ ）
- 質指標 $\text{mean log}(1+\text{FWCI})$ はほぼ互角（ $W/M \approx 1$ ）
- 男女差は主に top-1%（右裾）に集約 — 平均を見ていたら見落とす

参入年（コホート）を揃えて比べる

同じ年に始めた男女だけを比較し、参入時期の交絡を除く



ノートブック出力: エントリー年コホート別の 論文数・top-1%・mean log(1+FWCI)

慎重に読む

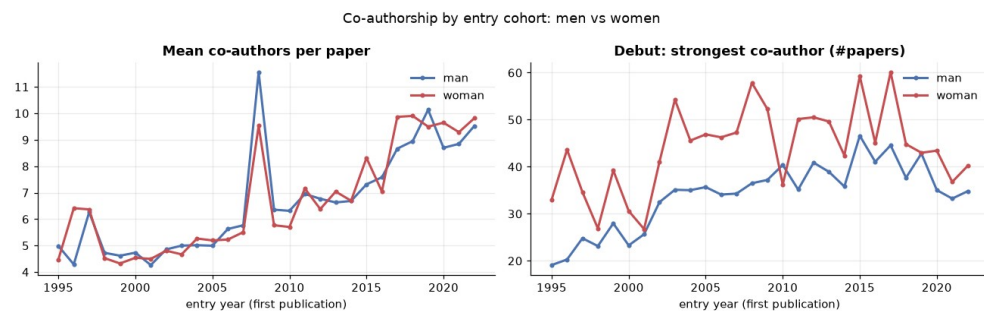
- 同年参入で揃えると、質指標 (top-1%・log 平均) は男女で同等～女性が上回る年も多い
- 累積差の一部は「参入時期の交絡」で説明できる — が、全部を説明できるわけではない
- 「差が消えた」と即断は禁物: unknown 多数・papers $\geq$ 2 の選択・記述統計にすぎない

コネクションと、漏れるパイプライン

デビュー時の有力者つながりと、参入後の継続率

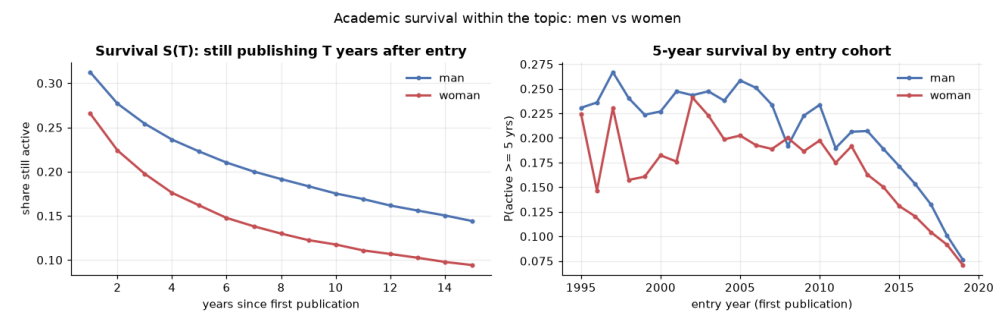
④ 共著（エントリー年別）

- 共著者数も最強共著者も、どの参入年でも女性が同等～上
- → 早期コネクション優位は参入時期では説明できない



⑤ 生存率 $S(T)$

- 参入から T 年後も出版している割合
- W/M が時間とともに低下 = 漏れるパイプライン



何が男女差を生むのか — まだ分からないことが多い

「キャリアの短さ」だけで片付けず、社会関係資本（ネットワーク）に目を向ける

- コホートや在籍長を揃えると累積差の多くは縮む
 - だが「差が消える／キャリアの短さが原因」と断定はできない
- 本チュートリアルは記述統計であり、名前ベース推定・選択バイアス・因果不明という限界がある
- 05 §4 の発見が重要な手がかり
 - 女性は同じ参入年でも、より有力な共著者となつがっていた
 - = 差は「能力」ではなく、共著ネットワーク＝社会関係資本の問題かもしれない

関連研究

Li, Zhang, Zheng, Cranmer & Clauset (2022, Nature Communications)

“Untangling the network effects of productivity and prominence among scientists”

- 中堅研究者の生産性・prominenceの男女差は、共著ネットワークの違いで大きく説明できる
- ネットワークは社会関係資本として先輩→後輩へ移転し、加齢とともに効果が減衰
- 所属機関の威信による優位も同様に説明できる

引用: Li, W., Zhang, S., Zheng, Z., Cranmer, S. J., & Clauset, A. (2022). Untangling the network effects of productivity and prominence among scientists. Nature Communications, 13, 4907.

06

NOTEBOOK 06

Disruption Index (革新性・断絶性)

How disruptive is a paper?

ある論文の革新性を「それ以前の文脈と断絶する度合い」として引用構造から測る。事前計算済みのエッジリストで、サンプリングせず全件を計算する。

Disruption Index をめぐる研究

革新性の指標として提案され、大きな議論を呼んだ

起源 — Funk & Owen-Smith (2017)。 技術変化を測るために提唱された CD 指標に始まります。ある成果が「それ以前の文脈と断絶する度合い」を、後続論文が何を引用するかという構造から捉えます。

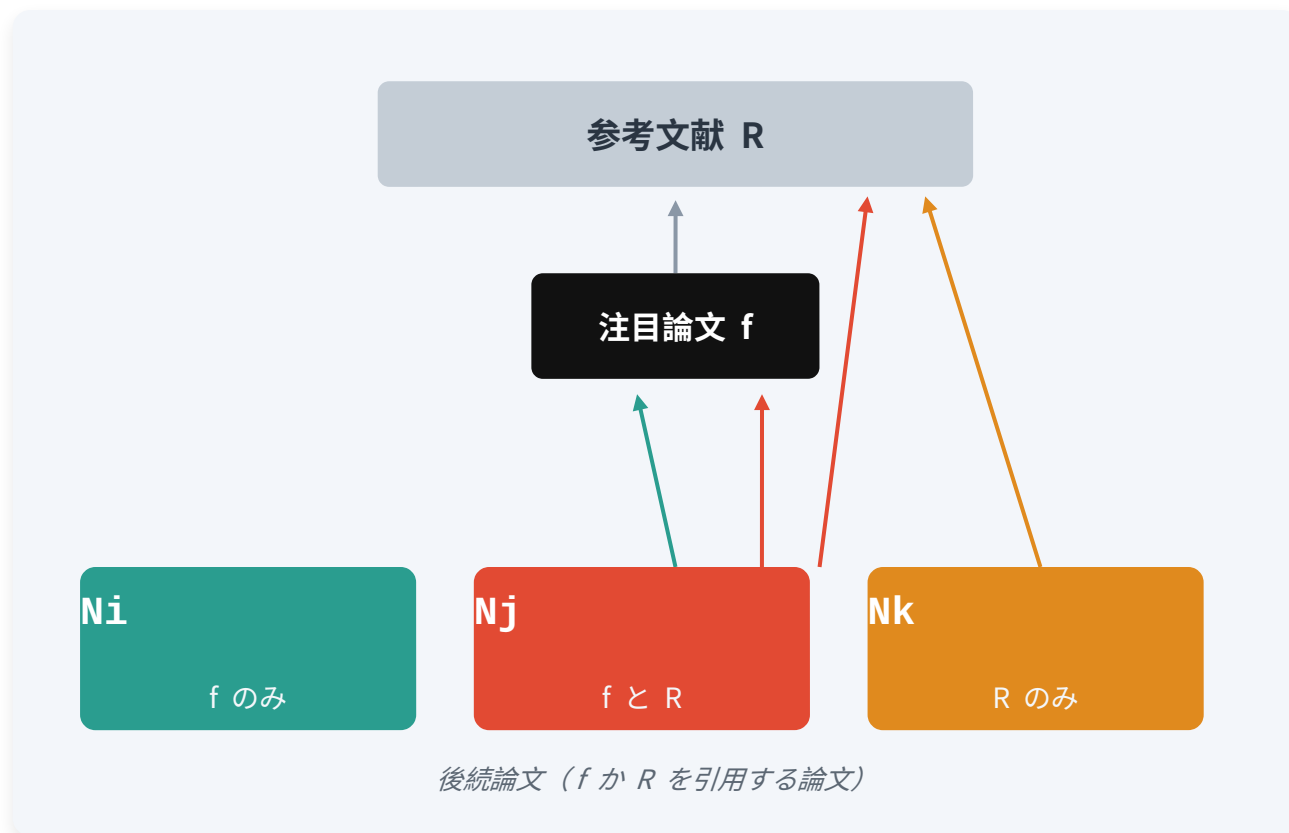
科学への導入 — Wu, Wang & Evans (2019, Nature)。 6,500 万件を超える論文・特許・ソフトウェアを分析し、小さいチームは新しい方向を切り拓き（disruptive）、大きいチームは既存を発展させる（developmental）という対比を示しました。

大きな論争 — Park, Leahey & Funk (2023, Nature)。 1945 年から 2010 年にかけて論文・特許の disruptiveness が大きく低下していると報告し、科学が「破壊的」から「統合的」へ向かっているという主張で議論を呼びました。

慎重論 — Leibel & Bornmann (2024)。 DI は引用窓・データ被覆・単一引用論文に敏感で、とりわけ時系列での比較には注意が必要だと指摘しています。値の解釈は目的に照らして慎重に行う必要があります。

DI の定義 — 後続論文を 3 群に分ける

注目論文 f と参考文献群 R を、後続論文がどう引くかを数える



DI の式

$$DI = (N_i - N_j) / (N_i + N_j + N_k)$$

正に大きい

disruptive (革新的・断絶的) : f だけが引かれ、過去の文脈は引かれなくなる

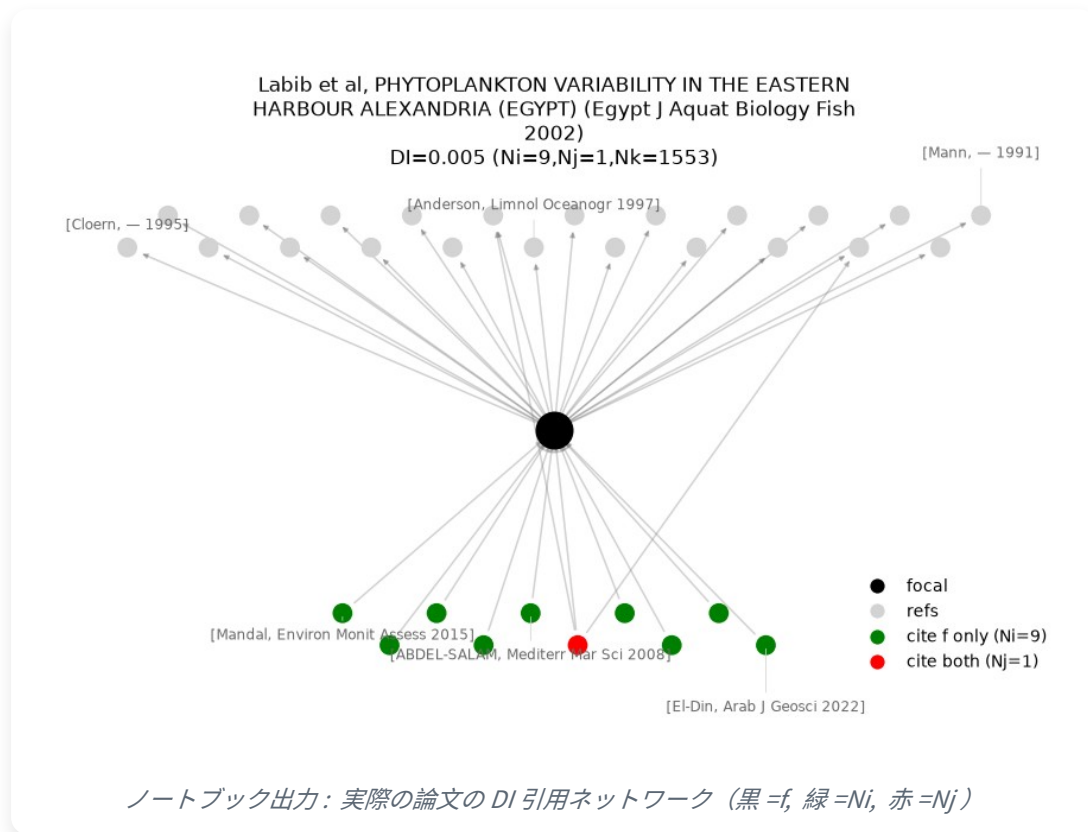
負に大きい

developmental (発展的・継承的) : f と参考文献がセットで引かれ続ける

1 本の論文の引用ネットワークを描く

DI を決める 3 群を色分けして、指標の意味を体感する

- 注目論文 f を中央、参考文献を上、引用する後続論文を下に配置
- 色分け
 - 黒 = 注目論文 f / 灰 = 参考文献そのもの
 - 緑 = f のみ引用 (N_i)
 - 赤 = f と参考文献の両方を引用 (N_j)
- 緑が多い ($N_i \gg N_j$) ほど DI は高く、その論文は「断絶的」



集合演算で DI を全件計算する

エッジリストを使うので、論文ごとに API を叩かずに済む

DI (集合演算)

```
def disruption(node):  
    # f の参考文献  
    node_r = set(citations_gb.target.loc[node])  
    # 参考文献を引用する後続論文  
    node_r_c = set(flatten(  
        [citations_gb.source.loc[x]  
          for x in node_r])) - {node}  
    # f を引用する後続論文  
    node_c = set(citations_gb.source.loc[node])  
    N_i = len(node_c - node_r_c)  # f のみ  
    N_j = len(node_c & node_r_c)  # 両方  
    N_k = len(node_r_c - node_c)  # 参考文献のみ  
    allnodes = node_c | node_r_c  
    return (N_i - N_j) / len(allnodes)
```

- 引用グラフをあらかじめエッジリスト化 (data/supplementary/)
- citations_gb: source→ 引用先 / target→ 引用元 を groupby で用意
- **per-paper API が不要なので、サンプリングせず全件を計算できる**
- この点だけ、01 のコーパスとは独立した自己完結ノート
- DI 分布は 0 付近に集中し、両裾に広がる

高 DI / 低 DI の例、100 万件で再現

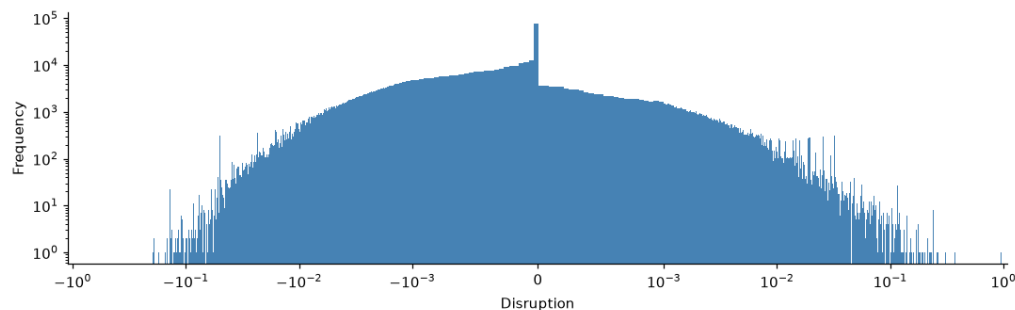
上位・下位の論文を確認し、Wu の公開データで分布を描く

高 DI (disruptive)

過去を断絶させた革新的論文。以降の研究が「f だけ」を引き、参考文献群を引かなくなる

低 DI (developmental)

既存の文脈を発展させた継承的論文。以降の研究は f と参考文献を「セット」で引く



ノートブック出力: Wu(2019) 100 万件の DI 分布 (symlog)

分布の読み方

- 0 付近に集中し、両裾に広がる
- 大多数の論文は革新性が中庸
- ごく一部が強く disruptive/developmental

DI が明らかにしたこと、そして論争

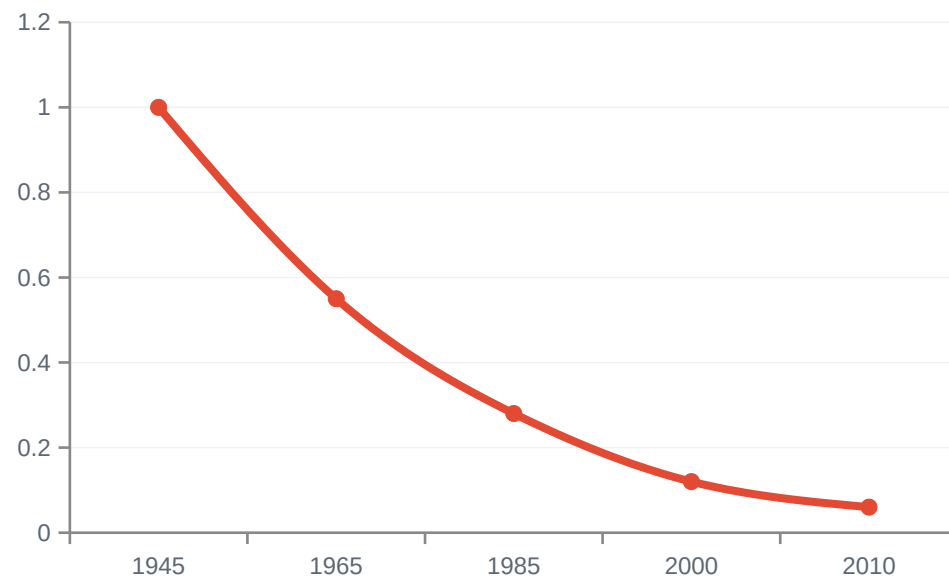
革新性の指標は科学の大きな傾向を可視化し、同時に議論を呼んだ

小さいチームは破壊する。 Wu, Wang & Evans (2019) は 6,500 万件超の論文・特許・ソフトウェアで、小さいチームが新しい方向を切り拓き、大きいチームが既存を発展させることを示しました（次スライドで再現します）。

破壊性は長期的に低下している。 Park, Leahey & Funk (2023) は、1945 年から 2010 年にかけて論文・特許の disruptiveness が大きく低下したと報告しました（右図はその傾向の模式化）。分野やチームサイズを調整しても残るとされ、大きな議論を呼びました。

解釈には注意が要る。 Leibel & Bornmann (2024) は、DI が引用窓やデータ被覆に敏感で、時系列比較の解釈には慎重さが必要だと指摘しています。

概念図：平均 disruptiveness の長期低下

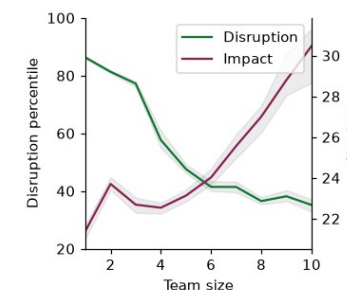


Park et al. (2023) の傾向を模式化（実データではない）

小さなチームほど disruptive

Wu et al. (2019) の主要結果を再現する

- 著者数 1 ～ 10 人ごとに DI と被引用の平均を取る
 - DI はパーセンタイル表示、灰色はブートストラップ 95%CI
- 小さなチームほど disruptive（革新的）
- 大きなチームほど developmental（発展的）
 - どの被引用帯でも、小チームほど DI が高い傾向



ノートブック出力: チームサイズ vs DI・被引用

⚠ DI の留意点

時系列比較には注意が必要（Leibel & Bornmann 2024）。データ被覆・引用窓・分野により値が変わる。目的に照らして慎重に使う

このチュートリアルで身につく手法

6 ノートを通して触れる、科学計量学の工具箱

データ取得。 REST API の filter・select・cursor、group_by による集計、sample や OR フィルタでの一括取得、そして parquet による基盤テーブル設計。

統計。 ジニ係数とローレンツ曲線による不平等の定量化、Clauset 法によるべき乗則の検定、分野正規化した FWCI、KS 検定や二項信頼区間といった推測統計。

機械学習・自然言語処理。 TF-IDF と OpenAI 埋め込み、KNN による近傍検索、Random Forest による予測、N-gram 抽出、GPT による要約。

ネットワークとテキスト。 直接引用グラフの構築、Leiden コミュニティ検出、UMAP による 2 次元化、KMeans、そして分離度を測る silhouette。

科学計量学の経験則。 引用の極端な偏り、ランダムインパクトルール、国際共著とインパクトの関係、そして Matthew / Matilda 効果。

革新性の指標。 Disruption Index の定義と計算、チームサイズ効果、そして時系列比較にひそむ落とし穴まで。

主な関連論文

各ノートで踏まえた代表的な研究

- Clauset, Shalizi & Newman (2009). Power-law distributions in empirical data. SIAM Review 51(4).
- Alstott, Bullmore & Plenz (2014). powerlaw: a Python package for heavy-tailed distributions. PLOS ONE.
- Sinatra, Wang, Deville, Song & Barabási (2016). Quantifying the evolution of individual scientific impact. Science.
- Traag, Waltman & van Eck (2019). From Louvain to Leiden: guaranteeing well-connected communities. Scientific Reports.
- McInnes, Healy & Melville (2018). UMAP: Uniform Manifold Approximation and Projection.
- Clauset et al. (2022). Untangling the network effects of productivity and prominence among scientists. Nature Communications 13, 4907.
- Funk & Owen-Smith (2017). A dynamic network measure of technological change. Management Science.
- Wu, Wang & Evans (2019). Large teams develop and small teams disrupt science and technology. Nature.
- Park, Leahey & Funk (2023). Papers and patents are becoming less disruptive over time. Nature.
- Leibel & Bornmann (2024). What do we know about the disruption index? Scientometrics.

このコーパスの限界と、全データ使用のすすめ

1 トピックに絞ったからこそ学びやすい。だが結論の一般化には注意が要る

限られたコーパスの限界

- 1 トピック（約 11 万件）だけを見ている
- 各研究者は「部分キャリア」しか見えない（他分野の業績が欠落）
 - ランダムインパクトルールが前半に偏る（§3-1）
 - H-index は「サンプル内」の下限值にすぎない
 - 生存率は分野転向を「離脱」と誤認しうる
- 直近年は引用が未成熟（to_publication_date で上限）
- DI は引用窓・データ被覆に敏感で時系列比較は要注意
- ジェンダーは名前ベース推定 + papers ≥ 2 の選択が入る

全データ・全業績で裏取りする

- 著者の全業績を使う（§3-2/01 付録 career）→ 部分キャリア・年齢バイアスが緩和
- OpenAlex 全体（約 2.6 億件）へスケール：cursor・OR 一括・group_by で全件処理
- 06 のように事前計算エッジリストで全件 DI を計算
- 複数トピック・分野横断で交絡（テーマ規模・資金）を評価
- 各時点のスナップショットがあれば、真の将来予測・時系列比較が可能に
- トピック限定は「学ぶ・試す」に最適。主張の一般化は全データで確認を

次のステップ

- まず 01 を実行してデータを用意し、02 ～ 06 へ進む（事前計算データは自動 DL）
- NAME='qc' に変えて、量子情報（T10020）で全分析を再現してみる
- OPENAI_API_KEY を設定すると、埋め込み KNN ・ GPT 要約 ・ クラスタ命名が有効に
- 自分の関心トピックの ID を DATASETS に足せば、任意の分野へ応用できる
- 留意：ジェンダー推定や DI の時系列比較など、手法の限界を常に意識する

github.com/asatani/sciscitutorial